

Strict Evaluation of Proposed EPC Intelligence Architecture

Executive Summary

The **EPC-Intel** proposal envisions a sophisticated multi-agent, RAG-driven AI platform for data-center EPC project delivery. It employs a React/Next.js frontend, FastAPI backend microservices, Celery workers, a PostgreSQL+pgVector datastore, and multiple specialized AI agents (compliance, scheduling, supply chain, commissioning). While the vision is ambitious, our analysis finds **significant over-engineering and high risk** in this design. The multi-agent, knowledge-graph-style architecture introduces severe complexity and points of failure, especially for a 48-hour hackathon with limited resources. Empirical evidence and expert analyses warn that such agentic systems frequently **fail due to coordination and verification gaps** [3†L150-L159] [19†L139-L148] . At the same time, simpler vector-based RAG approaches often suffice for most queries [5†L217-L225] [6†L331-L337] .

In short, the current design scores *poorly overall*: the weighted analysis yields a final score well below the 90+ needed to win. Key issues include brittle multi-service orchestration (Architecture & Scalability: 10/25), unnecessary AI agents (AI Usage: 5/15), unrealistic data assumptions (Data Feasibility: 3/10), and heavy deployment overhead (Deployment: 6/10). We recommend **dramatically simplifying the solution**: collapse redundant agents, favor rule-based logic where possible, and focus on a core RAG/chat frontend for project intelligence. Table [8] below compares the proposed multi-agent/knowledge-graph design with a simpler monolithic RAG+rule-based alternative.

Overall, we assign a **total score of 42/100**. This solution is **not** ready for a top-prize in a national hackathon. It assumes too much (e.g., “unlimited data and compute for all AI agents”) without evidence, and risks missing the deadline due to excessive scope. If time allowed, a phased roadmap is provided to recover by 90+ points, but even that requires drastic scope reduction and focused development of core features.

1. Architecture & Scalability (25 pts) — Score: 10/25

Justification: The architecture splits functionality into many microservices and Celery workers, which in theory aids modularity but risks catastrophic failure modes. Each AI agent and its backend service is a potential single point of failure: if the **Scheduler** agent or **Celery** queue hangs, the whole pipeline stalls. Real-world analysis shows multi-agent systems often “fail at high rates” due to poor coordination and lack of observability [3†L150-L159] . Even IBM notes that a *centralized* multi-agent system is vulnerable: if the central orchestrator fails, *all* agents fail [2†L232-L240] .

Moreover, **scalability is questionable**. The design uses PostgreSQL+pgVector as a stand-in for a vector DB, but relational databases are not optimized for high-volume semantic search. As data grows (thousands of documents, millions of chunks), query latency will balloon. Experts note that knowledge-graph traversals and vector lookups introduce latency and complexity [5†L217-L225] [6†L209-L217] . Unlike simple RAG

(one vector DB + LLM), this design requires heavy graph/agent coordination overhead that likely cannot be scaled in 48h.

Failure modes: Under load, the Celery queue or Redis broker could saturate, halting ingestion. The compliance and scheduling agents may compete for resources, causing timeouts. The central backend may become a bottleneck with dozens of concurrent LLM calls (each chat response could fire multiple agent queries). Single-instance deployment (on Render/Vercel) means no failover: any outage (e.g. redeploy error) crashes everything. The **architecture lacks clear isolation**: a bug in one service could cascade, contrary to the isolated service promise.

Remediation (priority, effort, impact):

- **[P1] Simplify Architecture:** Merge or remove non-critical agents. For example, combine RFI and Compliance into one "Project QA" service (Effort: *Low*, Impact: *High*). Remove the explicit knowledge graph layer unless justified by query complexity [5†L217-L225] .
- **[P2] Use a Single Vector DB:** Replace Postgres/pgVector with a specialized vector database (e.g. Qdrant, Weaviate) to handle scale and speed (Effort: *Medium*, Impact: *High*). This avoids heavy SQL tuning and latency.
- **[P3] Add Fault Tolerance:** Implement retries and fallbacks. For instance, if the Schedule agent fails, the system should still return other info. Use Redis in HA mode or abstract queueing to avoid one Redis point-of-failure (Effort: *Medium*, Impact: *Medium*).
- **[P4] Limit Concurrency:** Throttle simultaneous LLM and embedding requests to avoid spikes (Effort: *Low*, Impact: *Medium*).
- **[P5] Monitor & Circuit Breakers:** Add real-time health checks (e.g. circuit breaker on downstream LLM calls) so failures in external API calls don't cascade (Effort: *Medium*, Impact: *Medium*).

Unsupported assumptions:

- "The architecture can handle infinite documents" – no benchmarks or stress tests are provided.
- "All agents will coordinate perfectly" – contradicts findings that agentic AI often "goes off the rails" without strict oversight [3†L173-L182] [19†L123-L132] .
- "One Postgres DB is sufficient for both RAG and relational queries" – unrealistic at scale; expert opinion favors decoupling vector storage [6†L200-L207] [6†L331-L337] .

2. Technical Excellence & Correctness (20 pts) — Score: 12/20

Justification: The proposed implementation is well-structured on paper, but numerous technical gaps remain. Document ingestion uses PyMuPDF and Tesseract for OCR. Tesseract at 300 DPI is **error-prone** on complex drawings or low-quality scans; without image preprocessing or quality checks, OCR errors will pollute the vector store. The overlapping chunk logic (500-token chunks with 100-token overlap) is plausible, but risks splitting meaningful data (e.g. tables) across chunks. Embedding each chunk (using BGE or OpenAI) is correct, but storing millions of embeddings in Postgres is likely inefficient [6†L202-L210] .

The **Compliance Agent** combines LLM insight with deterministic rules. It is unclear how reliable the LLM comparisons will be; a single hallucinated inference could mark a valid spec as non-compliant or vice versa. Without extensive rule coverage, false positives are likely. Similarly, the **Schedule Risk model** (LightGBM)

has *no training data* mentioned. Throwing a model on synthetic features risks garbage-in→garbage-out; a simple rule-of-thumb or risk score might outperform an untrained ML model.

Their backend uses FastAPI and Celery – a solid stack. However, the architecture's complexity demands rigorous testing. Yet the Implementation Guide's testing strategy is generic and may not catch multi-service integration bugs. The lack of end-to-end testing (e.g. simulating an actual EPC dataset) is a notable gap.

Failure modes: Incomplete parsing could cause downstream failures (e.g. missing chunk triggers compliance checks). The chain of ingestion → embedding → vector store is sequential, so a failure in one (e.g. a corrupt PDF) could block all subsequent processes if not handled. Async tasks (Celery) may also lag, causing stale data. Edge cases like extremely large files may hit size/time limits. Potential race conditions (uploading docs while compliance agent is scanning) could cause inconsistent results.

Remediation:

- **[P1] Robust Parsing:** Incorporate multi-step OCR (e.g. image preprocessing with OpenCV for skew/crop) and fallbacks. Test on diverse sample docs (Effort: *Medium*, Impact: *High*).
- **[P2] Ground LLM with Rules:** Use the deterministic rule engine as the primary filter, reserving the LLM only for ambiguous cases (Effort: *Low*, Impact: *High*).
- **[P3] Unit & Integration Tests:** Add comprehensive tests for each module (parsing, compliance logic, API controllers) (Effort: *Medium*, Impact: *High*).
- **[P4] Data Validation:** Check parsed results against known patterns (e.g. if a spec lists ">=96%" but LLM outputs "<96%", flag for review) (Effort: *Medium*, Impact: *Medium*).
- **[P5] Deployment Dry-Runs:** Simulate full pipeline with mock data end-to-end before hackathon day to uncover integration issues (Effort: *High*, Impact: *High*).

Unsupported assumptions:

- "None of the AI models require GPUs" – this ignores that the LLM (even OpenAI API) has latency and cost. It also assumes inference speeds that may not hold under load.
- "Celery tasks always succeed" – failures in OCR or embedding could stall the queue if not retried.
- "LightGBM will yield meaningful schedule risk without data" – this claim lacks evidence and training data.

3. AI Usage & Justification (15 pts) — Score: 5/15

Justification: Many AI components appear chosen for their novelty rather than necessity. The **multi-agent** framing is fashionable, but as industry experts caution, "adding more agents often increases failure risk without better observability" [3†L150-L159] . In this proposal, tasks like compliance checking and RFI answering are handled by separate "agents," but a single well-designed retrieval-chatbot (RAG) could cover much of the knowledge assistant functionality. The Compliance Agent uses an LLM to compare specs vs. submittals, yet much of this is deterministic (e.g. numeric thresholds) and could be done with rule-checks. Over-reliance on LLM also risks hallucination; adding a knowledge graph or agent layer **does not eliminate hallucinations** [5†L137-L146] .

The **Knowledge Agent** is essentially RAG over documents, which is valid for Q&A. But if most queries are simple (lookup specs or past RFIs), vector search would suffice. As one expert notes, if the data size is small,

“it is more effective to just provide the whole context to the model instead of RAG” [5†L129-L137] – implying even the RAG layer might be overkill.

The **Supply Chain Agent** and **Commissioning Agent** use geospatial and standards logic. These could be largely rule-driven (alert if GPS ETA slip, check test values). No new ML model is clearly needed; framing them as “AI agents” adds complexity. The LightGBM schedule model is the one ML element; while ML can help predict delays, with no historical project data, its practical benefit is dubious.

Failure modes: Unnecessary AI components consume time and tokens (cost). For example, running an LLM to generate mitigation options could just repeat trivial logic. The multi-agent orchestration may suffer from the well-known “brittleness” where agents miscommunicate [3†L173-L182] . Also, the design may fall prey to the “Hallucination Trade-off”: chaining multiple LLM outputs (one for compliance, one for commissioning, etc.) compounds errors [5†L137-L146] .

Remediation:

- **[P1] Consolidate RAG:** Use a single conversational agent for open-ended queries, and reserve special services for rules only (Effort: *Low*, Impact: *High*).
- **[P2] Emphasize Rules Where Possible:** Hard-code compliance and QA criteria (especially numeric thresholds) as safeguards before LLM judgment (Effort: *Low*, Impact: *High*).
- **[P3] Conduct an Agent Cost-Benefit:** For each AI agent, clearly define success criteria. Drop agents whose value proposition is marginal (Effort: *Medium*, Impact: *High*).
- **[P4] Monitor LLM Use:** Track token usage per agent; cut or cache expensive prompts (Effort: *Low*, Impact: *Medium*).
- **[P5] Explainability Feature:** Provide transparency on why an LLM made a compliance decision (Effort: *Medium*, Impact: *Medium*).

Unsupported assumptions:

- “LLMs are needed for every check (e.g. specification compliance)” – experts warn deterministic rule engines often suffice for well-defined domains [19†L153-L162] .
- “More agents = better intelligence” – actually increases coordination load [3†L173-L182] .
- “Graph ensures correctness” – as noted, graphs alone don’t prevent AI from making errors [5†L137-L146] .

4. Data Strategy & Feasibility (10 pts) — Score: 3/10

Justification: The solution relies heavily on **industrial project data**: specs, schedules, vendor records, etc. In a hackathon, this data must be either provided (unlikely) or synthesized. No realistic source of labeled training data for schedule risk or procurement delays is available. The plan seems to assume generic metrics (lead times, RFI counts) can predict delays, but building a valid ML model requires *historic* project datasets, which are not mentioned.

Similarly, ingesting documents (PDF/DOCX) assumes they exist in digital form and can be reliably parsed. Many CAD drawings, vendor contracts, or test logs are complex or non-textual. The document pipeline may struggle with diagrams, non-English parts, or proprietary formats. We see no plan for data annotation or validation of OCR results.

Failure modes: Synthetic or default data models will likely underperform or mislead. For instance, a schedule-risk model trained on arbitrary numbers will produce meaningless outputs, leading to false alarms. Ingestion of low-quality scans could misclassify equipment specs, causing erroneous compliance flags.

The knowledge graph (if implemented) adds more data demands: ontologies and relationships need manual curation. This is labor-intensive and likely out of scope.

Remediation:

- **[P1] Curate Sample Datasets:** Secure or create a realistic, small dataset for each domain (e.g. an actual project schedule CSV, a sample RFI log) to test pipelines (Effort: *Medium*, Impact: *High*).
- **[P2] Scale Back ML:** Given data constraints, avoid training ambitious models. Prefer heuristic risk scores or even static thresholds (Effort: *Low*, Impact: *Medium*).
- **[P3] Validate Ingestion:** Run test suite of various document types (single/multi-column PDF, scanned images) and tune OCR/cleanup (Effort: *Medium*, Impact: *Medium*).
- **[P4] Incremental Data Build:** Plan for how to update vector DB and graph as new docs come in (Effort: *High*, Impact: *High*).
- **[P5] Define Data Governance:** Clearly note which data is “dummy” vs. real, and limit demo expectations accordingly (Effort: *Low*, Impact: *Low*).

Unsupported assumptions:

- “We have rich historical project data on hand” – not provided in the challenge.
- “AI will discover insights without explicit training” – leads to unpredictable outputs.
- “Ontology design can be done on-the-fly” – in reality it needs upfront domain expertise.

5. Deployment & Operations (10 pts) — Score: 6/10

Justification: The plan uses Docker, GitHub Actions, and managed services (Render, Vercel). For a prototype, this is technically solid, but in practice it's **overkill for 48 hours**. Setting up CI/CD, Prometheus/Grafana, Sentry, and multiple cloud components consumes scarce time. A minimal viable demo often skips production monitoring in favor of manual tests.

Operationally, the design risks complexity: 4 developers configuring Vercel, Render, MinIO, Postgres, Redis, RabbitMQ (implied by Celery), and LLM API access is a tall order. The document lays out phases, but any slip (e.g. Docker-compose mismatch, missing env var) could break the whole stack.

Furthermore, reliance on external services (Vercel, Render) introduces latency and potential costs. If the team uses the free tier, resource limits (CPU, memory, request quotas) may be hit by heavy RAG queries and image processing.

Failure modes:

- The deadline passes with partially deployed components.
- Unexpected downtime during demo (e.g. DB crashed).

- Lack of GPU: The architecture claims no GPU needed, but if a local LLM was used, it's false. Assuming cloud LLM (e.g. OpenAI) means being online and at mercies of rate limits and latency spikes.

Remediation:

- **[P1] Simplify Deployment:** Use **Docker Compose** locally instead of full cloud CI/CD. Skip Grafana/Loki in initial demo (Effort: *Low*, Impact: *Medium*).
- **[P2] Prioritize Features:** Delay non-essential services (like heavy monitoring). Focus on one stable deployment for core APIs (Effort: *Low*, Impact: *High*).
- **[P3] Resource Planning:** Pre-allocate test quotas for LLM API, and consider local LLM benchmarks to ensure timely replies (Effort: *Medium*, Impact: *Medium*).
- **[P4] Continuous Integration (Minimal):** Instead of full CI/CD, do nightly integration tests locally or on a single VM (Effort: *Medium*, Impact: *Medium*).
- **[P5] Fault Injection Tests:** Quickly test failure cases (kill Redis, overload Celery) to ensure controlled degradation (Effort: *Medium*, Impact: *Medium*).

Unsupported assumptions:

- "We can fully implement CI/CD and monitoring in a hackathon" – impractical in 48h.
- "Limited GPU is no issue" – building embeddings and running LLM queries fast requires either cloud GPUs or slow local inference, which is not addressed.
- "Render and Vercel will seamlessly scale to demo load" – even small spikes (from multiple judges) could break a free-tier service.

6. Business Impact & Innovation (10 pts) — Score: 5/10

Justification: The problem context is compelling: India's data-center boom and poor EPC performance is well-documented. If delivered, this platform *could* streamline compliance and risk management. However, the proposed solution's complexity undermines its business pitch. Judges may question: what is the **return on investment**? Without clear metrics or a crisp MVP, it reads as a collection of features rather than a focused product.

Innovation claims (multi-agent KG-based intelligence) are strong buzzwords but not proven. Many companies are already exploring RAG chatbots for project docs; the novelty here is the full EPC domain integration. Yet it's unclear how this solution leads to *measurable* business outcomes. For example, claiming "automated compliance checking" is only valuable if it cuts errors or inspection time – no estimates are given.

Remediation:

- **[P1] Highlight Use Cases:** Emphasize a key scenario (e.g. "reduce spec deviations by X%") with justification. Tie features to business pain (Effort: *Medium*, Impact: *High*).
- **[P2] Prototype Demonstration:** Build a limited demo (e.g. a chatbot over sample spec documents) that showcases the user experience, rather than all backend bells-and-whistles (Effort: *Low*, Impact: *High*).

- **[P3] Quantify Benefits:** Include even rough benchmarks or references (e.g. “Studies show AI can reduce rework by Y% in construction” from industry reports) **【3†L167-L175】** **【23†L333-L342】** (Effort: *Medium*, Impact: *Medium*).
- **[P4] Validate with Stakeholders:** If possible, get quick feedback from an EPC expert or target user to focus on the most valuable features (Effort: *High*, Impact: *High*).

Unsupported assumptions:

- “This platform will immediately yield ROI” – no business case is demonstrated.
- “All five suggested agents must be built” – overkill for an MVP; judges prefer a working core.
- “Enterprise adoption is straightforward” – integration with real EPC workflows (which are often paper-based) is non-trivial and not addressed.

7. Security & Compliance (5 pts) — Score: 2/5

Justification: Security is lightly touched. The architecture uses JWT-based auth and HTTPS (good baseline), but ignores data privacy concerns. EPC project documents often contain sensitive engineering or contractual data. RAG systems can inadvertently expose proprietary info to the LLM provider if not sanitized. There is no mention of encryption at rest (for S3/MinIO or Postgres) or auditing of data access. Additionally, compliance with regulations (e.g. data residency, copyright of standards) is not discussed.

Error handling is addressed (“global error handlers”), but GDPR/ISO standards for data protection in AI are missing. A notable oversight: LLM outputs could also reveal unintentional project information through hallucination.

Remediation:

- **[P1] Data Encryption:** Ensure all stored documents and embeddings are encrypted at rest and in transit (Effort: *Low*, Impact: *High*).
- **[P2] Privacy Filters:** Implement PII detection/redaction on documents before ingestion (Effort: *Medium*, Impact: *Medium*).
- **[P3] Least Privilege:** Assign minimal database permissions to each service (Effort: *Low*, Impact: *Medium*).
- **[P4] Compliance Checks:** Add logging for data access and changes, to support audit trails (Effort: *Medium*, Impact: *Medium*).
- **[P5] Review External APIs:** If using any external LLM API, vet their data usage policies; ensure no leaks (Effort: *Low*, Impact: *Medium*).

Unsupported assumptions:

- “Our hackathon MVP is exempt from security” – this is risky even for demos, as leaks can embarrass teams.
- “JWT and TLS is sufficient” – neglects data governance (Edinburgh Data Protection Board warns that LLMs can violate privacy rules if mishandled **【27†L11-L14】**).
- “No one will misuse the system” – access control (role-based) is mentioned, but not enforced in depth (e.g. internal vs. external users).

8. UX & Developer Experience (5 pts) — Score: 3/5

Justification: The frontend stack (Next.js, Tailwind, shadcn/ui) is modern and should yield a decent UI. The design of server vs client components is thoughtful. However, juggling many pages (dashboards, compliance, chat, commissioning) in 48h is ambitious. Developer productivity may suffer from the complex setup (monorepo, multiple module directories). Onboarding four developers to a new codebase with diverse tech (React, FastAPI, Celery) is non-trivial; conflicts may arise. The plan lacks details on component reuse or UI prototypes.

From a user standpoint, too many features at once could confuse judges. A simpler, polished chat interface might impress more than a half-built suite of dashboards.

Remediation:

- **[P1] Design Iteration:** Prioritize a minimal, intuitive interface flow (Effort: *Low*, Impact: *High*). E.g., a single dashboard plus chat window.
- **[P2] Style Guide:** Enforce consistent UI components (shadcn provides many) to speed up development (Effort: *Low*, Impact: *Medium*).
- **[P3] Developer Docs:** Write clear READMEs and API specs so all devs align (Effort: *Low*, Impact: *Medium*).
- **[P4] Component Library:** Extract shared UI elements early (Effort: *Medium*, Impact: *Medium*).
- **[P5] Demo Modes:** Include a “sample data” toggle to easily show features without manual input (Effort: *Low*, Impact: *High*).

Unsupported assumptions:

- “UX will ‘just work’ under time pressure” – teams often underestimate UI polish time.
- “Developers can parallelize easily” – monorepo merge conflicts or inconsistent tech choices (JSX vs TS, etc.) can slow progress.
- “All intended user flows fit into planned pages” – ad-hoc changes are likely; a rigid UI plan may need more flexibility.

Overall Score: 42/100 (*Weighted total*)

The above weighted subscores yield **42/100**. Key deficits are in scalability, AI justification, and data feasibility. This architecture is **not ready for top-prize consideration** in its current form.

Final Verdict: Fail for a national hackathon top prize. While the vision is impressive, the execution plan is overambitious. Judges in such settings favor *solid, demonstrable functionality* over speculative innovation. Without drastic simplification (e.g., focusing on a single core feature well), the solution risks appearing unfinished and may not be achievable within constraints 【3†L148-L157】 【19†L123-L132】 .

6-Phase Implementation Roadmap (to achieve ≥ 90)

To salvage this into a winning prototype, we recommend a strict 6-phase plan, each spanning 8 days (assuming 4 devs, **48 total days** to simulate intense development, given the complexity). Each phase builds toward a Minimum Viable Product:

Phase	Timeline (Days)	Focus
Phase 1: Core Setup & Data Prep Days 1–8	- Finalize simplified architecture: merge agents - Setup monorepo, Docker, and essential infra (DB, MinIO). - Curate realistic sample data (specs, schedules, RFIs). - Implement core document ingestion pipeline (PyMuPDF + OCR) and vector storage.	
Phase 2: Compliance & RAG Chat Days 9–16	- Develop the Compliance & QA rules engine (no LLM) with sample data. - Build a basic RAG-based chat interface over ingested docs (using a vector DB). - Validate with a few canned queries.	
Phase 3: Simplified Scheduling Module Days 17–24	- Replace ML model with heuristic risk scoring for schedule (e.g. critical path + slack). - Build API endpoints for schedule upload and risk alerts. - Integrate schedule info into the RAG chat (simple retrieval).	
Phase 4: Deployment & Testing Days 25–32	- Deploy combined backend+frontend on a single platform (Docker Compose or Render). - Conduct end-to-end tests with stakeholders (if available). - Refine UX flows based on feedback.	
Phase 5: Enhancements & QA Days 33–40	- Add minor AI features (e.g. a small fine-tuned LLM snippet for complex query, if time). - Improve UI polish (charts, maps optional). - Write thorough unit/integration tests.	
Phase 6: Final Integration & Demo Prep Days 41–48	- Final stress tests (simulate multiple docs, users). - Fix bugs, ensure scalability tweaks (indexing, caching). - Prepare demo script, visuals (diagrams), and executive summary. - Dry-run presentation.	

Each phase assumes focus on *fewer, higher-impact* components. Key dependencies: complete Phase 1 is prerequisite for all; Phase 2 and 3 build on the data and DB. By Day 48, aim for a functional end-to-end demo covering the core promise: **integrated document intelligence & risk alerts**.

Alternative Architectures Comparison

Architecture	Cost & Complexity	Time to Prototype	Scalability	Judge Appeal
Monolithic RAG + Rules	Low – Single codebase, minimal infra (vector DB + LLM). Easy to manage.	Fast (Few days) – Build one FastAPI + React app.	Moderate – Simple to deploy, but limited by single instance. Scaling is coarse (scale whole app).	Moderate – Less “wow” factor, but high reliability. Judges see working demo.
Full Multi-Agent + KG	High – Many microservices, KG, vector store, complex CI/CD. High dev effort.	Very Slow (weeks/months) – Requires learning and integrating many components.	High if well-built – Can scale services separately, but risk of coordination issues [3†L150-L159] .	Mixed – Sounds innovative; judges may be impressed if working, but harsh if flaky.

Table 1. Comparison of a **simpler monolithic** approach versus the proposed **multi-agent/knowledge-graph** approach, across key factors. The monolithic design is cheaper and quicker to build but less extensible, whereas the multi-agent/KG architecture is expensive and complex, and risks missing the hackathon deadline due to its complexity [26†L988-L997] [5†L217-L225] .

Diagrams

Figure 1. High-Level Architecture (Current Multi-Agent Design):

```

flowchart LR
    subgraph UI
        A["User<br/>(Project Manager, Engineer)"]
        A -->|HTTP/HTTPS| B["Frontend<br/>(React/Next.js)"]
    end
    B -->|REST API| C["Backend<br/>(FastAPI)"]
    C --> D["Document Storage<br/>(MinIO/S3)"]
    C --> E["Database<br/>(PostgreSQL + pgVector)"]
    C --> F["Authentication Service"]
    C --> G["Compliance Service"]
    C --> H["Schedule Service"]
    C --> I["Supply Chain Service"]
    C --> J["Commissioning Service"]
    C --> K["RFI/Q&A Service"]
    C --> L["Knowledge Graph (Neo4j?)*"]
    D -->|Upload/Fetch| G
    D --> H

```

```
E --> G
E --> H
E --> I
E --> J
E --> K
L --> G
L --> H
L --> I
L --> J
L --> K
click K href "#AI-Agent" "Multi-Agent Coordination"
```
```

\*(Diagram: The multi-agent EPC-Intel platform. Users interact via a web UI. The FastAPI backend orchestrates multiple specialist services (compliance, scheduling, supply chain, commissioning, RFI). A document store and vector database feed each agent. A central knowledge graph integrates data. As noted by experts, such distributed architectures introduce many points of failure and coordination overhead [3†L150-L159] [26†L988-L997] .)\*

**Figure 2. Simplified Monolithic RAG+Rules Design:**  
mermaid

```
graph TD
 A[User] --> B[Single Frontend App]
 B --> C[Single Backend API]
 C --> D[Vector DB + RAG Engine]
 C --> E[Relational DB]
 C --> F[Rule-based Engines
(Compliance & Scheduling)]
 C --> G[LLM API]
 G --> C
```
```

(Diagram: A leaner alternative. One backend handles user requests, with a unified vector database for retrieval and LLM calls. Compliance and scheduling checks are primarily rule-based modules. This reduces service count, speeding development and reducing failure modes [12†L829-L837] [5†L217-L225] .)

Sample Scoring Rubric (Illustrative)

Criterion	Weight (%)	Excellent (5 pts)
Poor (0 pts)		
-----	-----	-----
Architecture & Scalability	25%	Clear, simple design; scales and tolerates faults
Modular design with minor single-fail points		Overly complex; single point of failure

Technical Correctness	20%	Well-tested, correct implementations	Basic functionality but some bugs	Bugs/missing features; unreliable
AI Usage & Novelty	15%	Justified AI use; innovative method	Standard use of AI components	Unnecessary or misused AI
Data Strategy	10%	Data sources ready; well-prepared	Some data concerns; mitigations proposed	No plan for data; unrealistic assumptions
Deployment & Ops	10%	Robust deployment and monitoring	Basic deployment; some manual steps needed	No deploy plan; untested operations
Business Impact	10%	Clear ROI; addresses real needs	Some impact identified; partial solution	No clear value; over-engineered
Security & Compliance	5%	Strong data protection, compliance controls	Basic auth/logging	Sensitive data exposure; missing policies
UX & Dev Experience	5%	Intuitive UI; clear docs; smooth dev workflow	Usable UI; some missing docs	Cluttered UI; developers unclear on next steps

Table 2. An example rubric. Scores in this review reflect issues in nearly every category: points were lost for overcomplexity, weak AI justification, and lacking data/deployment planning.

Prompt for LLM-Based Review

You are an expert AI systems judge evaluating a proposed architecture for a data-centre EPC intelligence platform. Critically score this solution (0–100) across categories: Architecture & Scalability, Technical Correctness, AI Usage, Data Strategy, Deployment, Business Impact, Security, and UX. For each: provide a numeric score, cite concrete failure modes or research where applicable, list key issues, and recommend prioritized fixes (with effort/impact). Identify unsupported claims or assumptions. Conclude with an overall weighted score and a pass/fail verdict. Finally, propose a brief multi-phase roadmap (with timelines) to make the solution award-worthy. Be concise but thorough and evidence-driven. ""